

CODING & SOLUTION

Project Name

ComicCraft – AI Powered Comic Story Generator

Team ID: _____

Solution Summary

Field	Details
Repository Link / URL	https://github.com/your-team/comiccraft
Programming Language(s)	Python, JavaScript, HTML5, CSS3
Framework(s) Used	FastAPI, Jinja2
Database	PostgreSQL, SQLite
AI Technologies	Google Gemini API, Hugging Face Inference API
Image Processing	Pillow
PDF Generation	FPDF
Version Control	Git, GitHub
Deployment Platform	Docker, Render / AWS

Key Features Implemented

User Management

- User Registration
- User Login
- User Authentication
- User Profile Management

Story Generation

- AI-powered comic storyline generation
- Dialogue generation
- Narration generation
- Scene description generation

Comic Generation

- Multi-panel comic creation
- Automatic comic sequencing
- Character-driven storytelling

Image Generation

- AI-generated comic illustrations
- Character image generation
- Scene visualization
- Art style customization

Comic Layout Engine

- Automatic comic panel arrangement
- Story and image integration
- Comic preview generation

Export Features

- PDF comic generation
- Downloadable comic files
- Comic history management

User Interface

- Responsive design
 - Interactive comic creation workflow
 - Preview functionality
-

Pending / Future Features

Future Enhancements

- Multi-language support
 - Voice narration
 - Animated comic generation
 - Mobile application
 - Social media publishing
 - Real-time collaboration
 - Character memory system
 - Cloud storage integration
-

Setup / Run Instructions

Step 1 – Clone Repository

git clone <https://github.com/your-team/comiccraft.git>
cd comiccraft

Step 2 – Create Virtual Environment

```
python -m venv venv
```

Activate Environment

Windows:

```
venv\Scripts\activate
```

Linux / macOS:

```
source venv/bin/activate
```

Step 3 – Install Dependencies

```
pip install -r requirements.txt
```

Step 4 – Configure Environment Variables

Create a .env file:

```
GEMINI_API_KEY=your_api_key  
HF_API_KEY=your_huggingface_key  
DATABASE_URL=postgresql://username:password@localhost/comiccraft
```

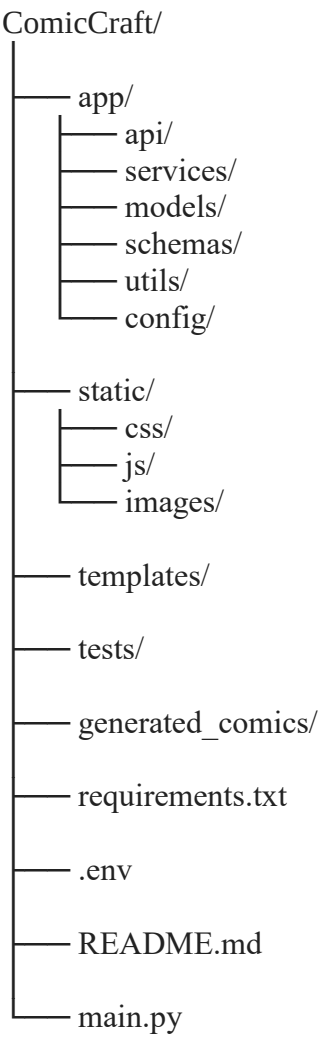
Step 5 – Run Application

```
uvicorn main:app --reload
```

Step 6 – Open Application

<http://localhost:8000>

Source Code Structure



Functional Requirement Coverage

Functional Requirement	Status
User Authentication	Completed
Prompt Input System	Completed
Story Generation	Completed
Dialogue Generation	Completed
Narration Generation	Completed
Image Generation	Completed
Comic Layout Builder	Completed
Comic Preview	Completed
PDF Export	Completed
Download Functionality	Completed

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions/classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments/documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	Application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments

Coding Standards Followed

- PEP 8 Python Coding Standards
- REST API Best Practices
- Modular Architecture
- Separation of Concerns
- Environment-Based Configuration

Security Practices

- JWT Authentication
- Secure API Communication
- Environment Variable Management
- Input Validation

Performance Optimizations

- Asynchronous API Calls
- Efficient Prompt Processing
- Optimized Image Handling
- Modular Service Architecture

Testing Performed

Unit Testing

- API endpoint testing
- Story generation testing
- PDF generation testing

Integration Testing

- Gemini API integration
- Image generation integration
- Database integration

User Acceptance Testing

- Comic generation workflow
- PDF export workflow
- User authentication workflow

Conclusion

ComicCraft successfully addresses the challenge of comic creation by integrating Generative AI technologies into a unified platform. The implemented solution enables users to generate complete comics from simple prompts, including storylines, dialogues, illustrations, and downloadable PDFs. The application follows clean coding practices, modular architecture, and modern software engineering principles, ensuring scalability, maintainability, and future extensibility.